U3 - Interrupts and Arithmetic for Computer

Data Hazards in RISC-V Pipelines

Introduction

A **data hazard** occurs when an instruction depends on the result of a previous instruction that has not yet completed all stages of the pipeline.

In a **five-stage pipeline**, the stages are:

```
IF → ID → EX → MEM → WB
```

> A hazard occurs when the data required in the **EX** or **ID** stage is still in progress in an earlier instruction's **MEM** or **WB** stage.

Example 1: Data Dependency

```
add x19, x0, x1
sub x2,  x19, x3
```

- The `sub` instruction needs the value of `x19`, which is written by the `add` instruction.
- The `add` result is only available at the **WB** stage.
- However, `sub` needs `x19` during its **EX** stage.

In form of a timing diagram, it looks something like this:

Cycle	add (x19 = x0 + x1)	sub (x2 = x19 - x3)	Data available?
1	IF		
2	ID	IF	
3	EX	ID	
4	MEM	EX	Needs x19 (not yet written)
5	WB	MEM	Result written now
6		WB	

> Without intervention, the pipeline would use an *incorrect* (old) value of `x19`.

Avoiding Hazards

There are two main approaches: **stalling** and **forwarding**.

1. Stalling (Pipeline Bubble)

The processor inserts idle cycles (“bubbles”) until the data becomes available.

Cycle	add	sub	Comment
1	IF		
2	ID	IF	
3	EX	ID	
4	MEM	STALL	sub waits for x19
5	WB	EX	x19 is now available
6		MEM	
7		WB	

> This is cool as stalling the pipelining ensures correctness but reduces performance (CPI - Clocks Per Instructions increases).

2. Forwarding (Data Bypassing)

Forwarding directly passes the result from one pipeline stage (like EX/MEM) to a later stage (EX of another instruction) before it’s officially written to the register file.

This avoids the need to stall.

Cycle	add	sub	Comment
1	IF		
2	ID	IF	
3	EX	ID	
4	MEM	EX	Forward result of `add` to `sub` EX
5	WB	MEM	
6		WB	

> Forwarding logic adds hardware complexity but improves throughput.

Example 2: Independent vs Dependent Instructions

Independent Instruction Set

```
lw  x10, 40(x1)
sub x11, x2, x3
add x12, x3, x4
lw  x13, 48(x1)
add x14, x5, x6
```

> Each instruction works with independent registers. The pipeline flows smoothly and thus no hazards.

Dependent Instruction Set

```
sub x2,  x1, x3
and x12, x2, x5
or  x13, x6, x2
add x14, x2, x2
sw  x15, 100(x2)
```

> Here, all instructions depend on `x2`, produced by the first `sub`.

Cycle	sub	and	or	add	sw	Notes
1	IF					
2	ID	IF				
3	EX	ID	IF			and waits for x2
4	MEM	STALL	IF			
5	WB	EX	ID	IF		x2 written back
6		MEM	EX	ID	IF	

> Stalls or forwarding would be required to ensure all instructions receive the correct updated `x2`.

TLDR

- **Data Hazard:** Occurs when an instruction depends on data not yet available.
- **Solutions:**
  - *Stalling:* Simple but costly as it adds additional clock cycles, so performance goes down.

- *Forwarding:* Efficient but requires additional hardware.

Detecting the Need to Forward

Forwarding is mainly needed when a value is required in the **EX stage** of one instruction while an earlier instruction is still in the **MEM** or **WB** stage, preparing to write that same value.

This typically occurs for ALU operations or address calculations (for load/store).

To handle this, the pipeline tracks register numbers as they move through the stages.

Notation	Meaning
`ID/EX.RegisterRs1`	Register number of source 1 for the instruction currently in the EX stage
`ID/EX.RegisterRs2`	Register number of source 2 for the instruction currently in the EX stage
`EX/MEM.RegisterRd`	Destination register number of the instruction in the MEM stage
`MEM/WB.RegisterRd`	Destination register number of the instruction in the WB stage

A **data hazard** is detected when:

```
EX/MEM.RegisterRd == ID/EX.RegisterRs1   (Hazard 1a)
EX/MEM.RegisterRd == ID/EX.RegisterRs2   (Hazard 1b)
MEM/WB.RegisterRd == ID/EX.RegisterRs1   (Hazard 2a)
MEM/WB.RegisterRd == ID/EX.RegisterRs2   (Hazard 2b)
```

Example: Detecting Hazards in the Pipeline

```
sub x2, x1, x3
and x12, x2, x5
or  x13, x6, x2
```

1. **First Hazard (1a)**
   - Between `sub` and `and`.
   - Detected when `and` is in **EX** and `sub` is in **MEM**.
   - `EX/MEM.RegisterRd = ID/EX.RegisterRs1 = x2`.
2. **Second Hazard (2b)**

- Between `sub` and `or`.
- Detected when `or` is in **EX** and `sub` is in **WB**.
- `MEM/WB.RegisterRd = ID/EX.RegisterRs2 = x2`.

Double Data Hazard

A **double data hazard** occurs when both the instruction currently in **MEM** and the instruction in **WB** are writing to registers that match the source operands of the instruction in **EX**.

For instance:

Stage	Instruction	Destination	Source(s)
MEM	`add x5, x1, x2`	x5	—
WB	`sub x7, x3, x4`	x7	—
EX	`and x8, x5, x7`	x8	Rs1=x5, Rs2=x7

Here, both operands needed by `and` are being written by different earlier instructions.

The forwarding unit must:

- Forward **x5** from the EX/MEM pipeline register.
- Forward **x7** from the MEM/WB pipeline register.

Hence, both **ForwardA** and **ForwardB** signals activate simultaneously: one from each source.

Forwarding Multiplexer Control Logic

Each ALU input has a **2-bit forwarding control signal**:

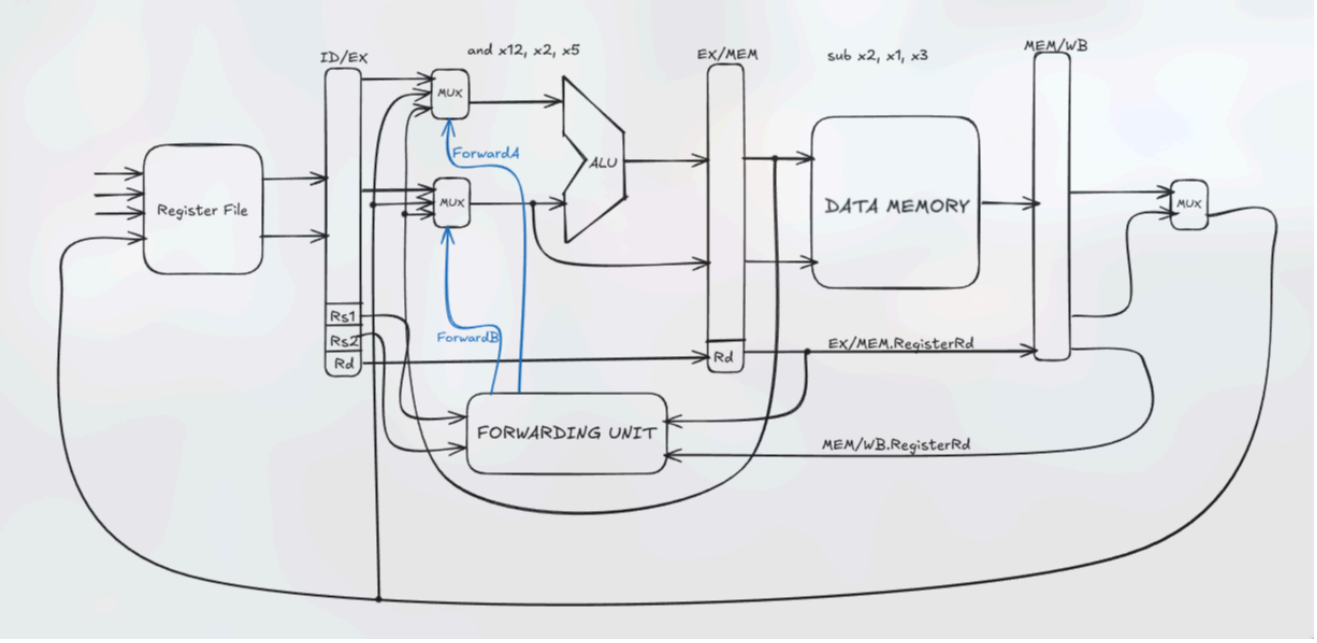
Control Signal	Source	Description
`ForwardA = 00`	ID/EX	First operand comes directly from register file
`ForwardA = 10`	EX/MEM	First operand forwarded from prior ALU result
`ForwardA = 01`	MEM/WB	First operand forwarded from data memory or earlier ALU result

Control Signal	Source	Description
`ForwardB = 00`	ID/EX	Second operand comes directly from register file
`ForwardB = 10`	EX/MEM	Second operand forwarded from prior ALU result
`ForwardB = 01`	MEM/WB	Second operand forwarded from data memory or earlier ALU result

---

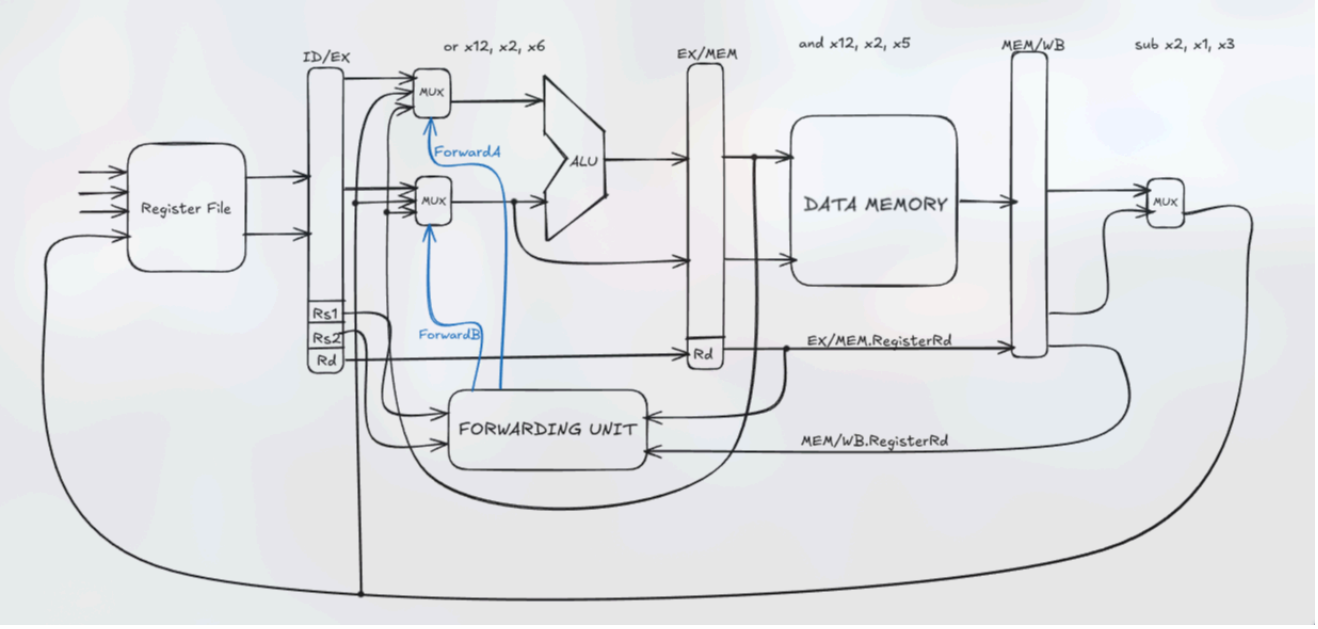
Forwarding Decision Conditions

1. EX Hazard (Forward from EX/MEM)




```
if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
        ForwardA = 10

if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
        ForwardB = 10
```

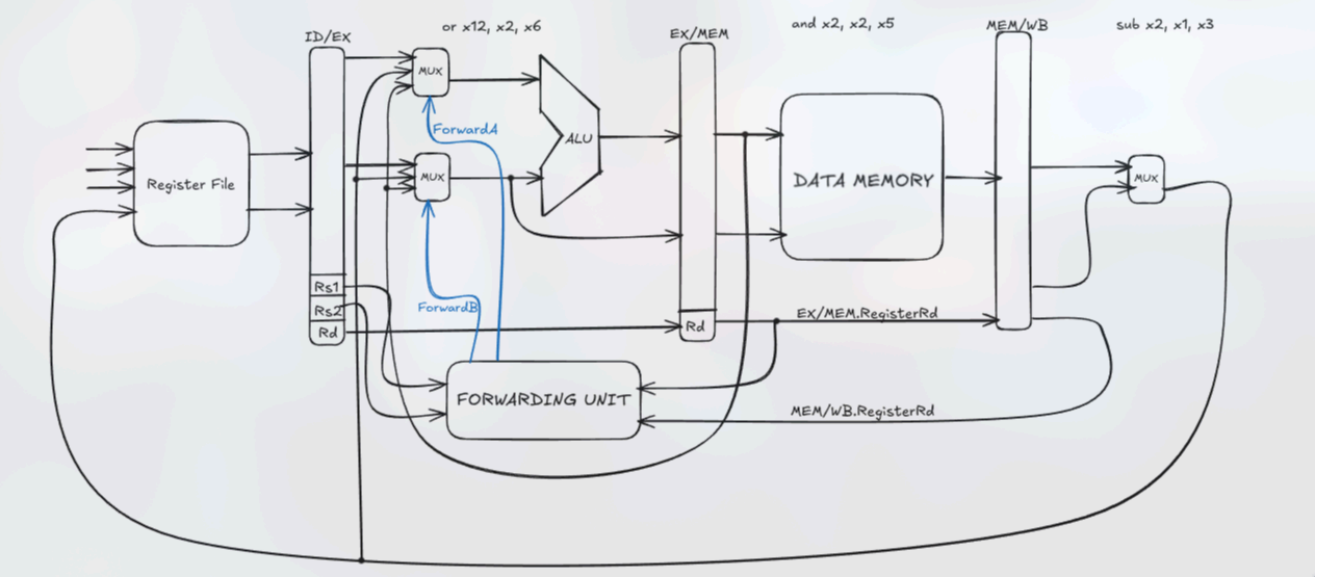
2. MEM Hazard (Forward from MEM/WB)




```
if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
        ForwardA = 01

if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
        ForwardB = 01
```

3. Priority in Double Data Hazards

When both hazards are true, **EX forwarding has priority** (since EX/MEM holds newer data).




```
if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd ≠ 0)
    and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
```

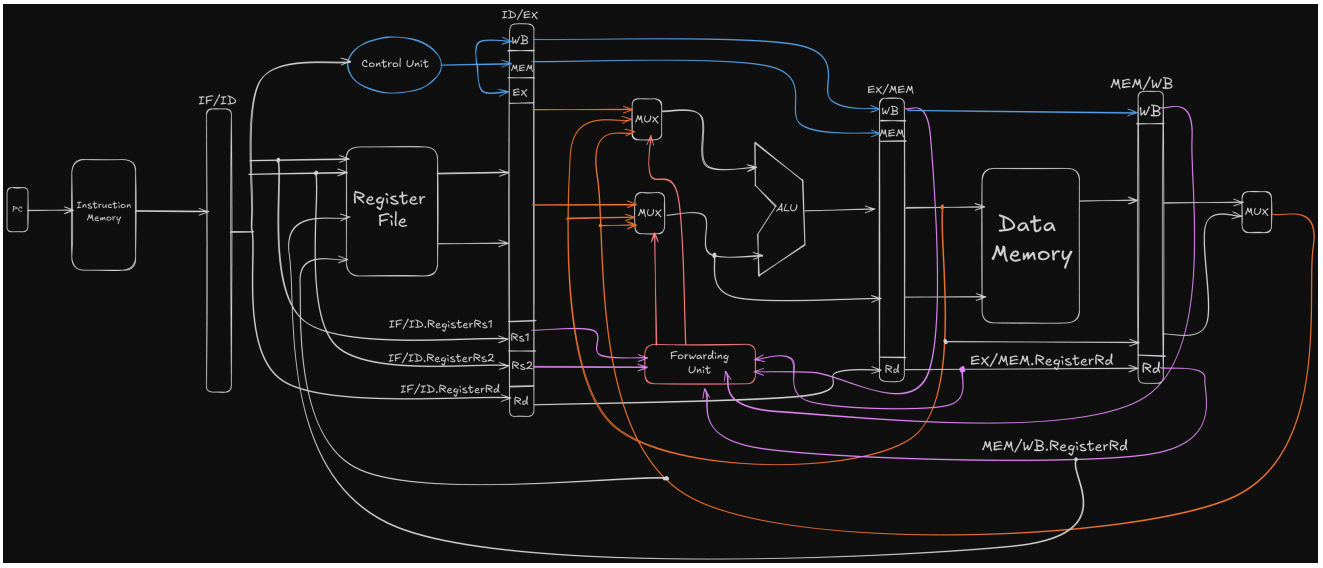
```
            and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
        ForwardA = 01

if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd ≠ 0)
    and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
        ForwardB = 01
```

This ensures that:

- If both MEM and WB could supply data, EX/MEM (newer result) is used.
- MEM/WB is used only when EX/MEM doesn't produce the required value.

TLDR

- Forwarding avoids pipeline stalls by directly routing intermediate results.
- The control logic determines whether to forward from **EX/MEM** or **MEM/WB** pipeline registers.
- Double data hazards require forwarding from both sources simultaneously.
- EX/MEM has priority over MEM/WB for the same register destination.



> The fully pipelined datapath including forwarding unit :D

Exceptions and Interrupts in RISC-V

Introduction

Exceptions and **Interrupts** are both *traps*: events that transfer control to a privileged trap handler.

- **Exception:** Synchronous event caused by the current instruction (e.g., illegal instruction, divide-by-zero).
- **Interrupt:** Asynchronous event triggered externally (e.g., timer interrupt, I/O signal).

When a trap occurs, control transfers to a **trap handler** whose address is determined by the `mtvec` register and the selected mode (direct or vectored).

Privilege Modes

RISC-V defines three privilege levels:

Mode	Description	Typical Use
Machine (M)	Highest privilege	Handles traps, manages hardware
Supervisor (S)	OS kernel	Manages virtual memory, exceptions
User (U)	Lowest privilege	Executes user applications

Each level has its own set of **Control and Status Registers (CSRs)** for managing traps and interrupts.

Trap Handling

When a trap occurs:

1. `mepc` (or `sepc`) stores the address of the instruction that caused the trap.
2. `mcause` (or `scause`) stores the reason or interrupt code.
3. `mtval` (or `stval`) may store additional faulting information (e.g., bad address).
4. `mtvec` holds the base address for the trap handler.
5. The processor sets `PC = mtvec.BASE` (or a vectored offset, depending on mode).

Trap Handling Modes

1. Direct Mode

All traps jump to the same base address:
Example:

```
Instruction: add x11, x12, x11
If hardware fault occurs:
→ mcause ← cause code
→ mepc   ← address of add instruction
→ PC     ← mtvec.BASE
→ Control passes to trap handler
```

2. Vectored Mode

The handler address depends on the **trap cause code**:

Examples (Base = 0x80000000):

Event	Code	Handler Address
Illegal Instruction	2	0x80000008
Machine Timer Interrupt	7	0x8000001C

Vectored mode enables faster response since the hardware jumps directly to the correct routine.

Key Control and Status Registers (CSRs)

Register	Purpose	Notes
`mtvec`	Trap vector base address	Determines Direct / Vectored mode
`mepc` / **`sepc`**	Address of instruction that caused trap	Used for returning via `mret` / `sret`
`mcause` / **`scause`**	Encodes cause of trap	MSB=1 → interrupt, 0 → exception
`mstatus` / **`sstatus`**	Global interrupt enable	Bit `MIE` / `SIE`
`mie` / **`sie`**	Enables specific interrupts	Bits: `MTIE`, `MSIE`, `MEIE`
`mip` / **`sip`**	Pending interrupt flags	Bits: `MTIP`, `MSIP`, `MEIP`
`mtval` / **`stval`**	Trap value register	Provides additional fault info

Interrupt Enable Summary

Level	Register	Bit	Description
Global	`mstatus`	`MIE`	Enables all machine-mode interrupts
Individual	`mie`	`MTIE` / `MSIE` / `MEIE`	Enables specific interrupt types
Pending	`mip`	`MTIP` / `MSIP` / `MEIP`	Shows pending interrupts

Example:

```
MTIE = 0 → Machine Timer Interrupt disabled
MEIE = 1 → Machine External Interrupt enabled
MEIP = 1 → External interrupt pending
```

Returning from Trap

To resume normal execution:

1. The handler restores registers and state.
2. Executes `mret` (or `sret`) to return:

Exceptions and Interrupts in Pipelined Implementations

Exceptions in Pipelines

In a **pipelined processor**, exceptions are treated as **control hazards**, just like branches.
Since multiple instructions overlap in execution, an exception in one stage can affect others.

Example scenario:

- Suppose the instruction `add x1, x2, x1` is in the **EX** (Execute) stage.
- The following two instructions are in the **ID** and **IF** stages.

If a **hardware malfunction** occurs during the `add`:

1. Prevent **x1** from being overwritten.
2. Complete all **previous** instructions (before `add`).
3. **Flush** the `add` instruction and all **younger** instructions (in ID and IF).
4. Set the **`sepc`** and **`scause`** registers.
5. Transfer control to the **trap handler** (just like a mispredicted branch).

This uses the **same control-path hardware** as branch handling, except the branch direction is replaced by an exception signal that **deasserts control lines** and initiates the trap.

Example: Exception Handling in Pipeline

```
    addi x1, x0, 10       # IF: Load immediate 10 into x1
    addi x2, x0, 0        # ID: Initialize x2 to 0
    div  x3, x1, x2       # EX: Divide by zero exception here!
    addi x4, x0, 5
    add  x5, x1, x4

# --- Trap Handler ---
Trap_Handler:
    csrr  x10, mepc       # Save address of faulting instruction
    csrr  x11, mcause     # Get cause of exception (divide-by-zero)
    addi  x12, x0, 1      # Mark exception as handled
    mret                  # Return from trap
```

Pipeline Stage	Instruction	Action
IF	`addi x1, x0, 10`	Fetched normally
ID	`addi x2, x0, 0`	Decoding
EX	`div x3, x1, x2`	Divide-by-zero detected → exception triggered
MEM/WB	—	Pipeline control logic flushes ID and IF stages
Trap	—	`mepc` = address of `div`, `mcause` = Divide-by-zero
Handler	—	Control transfers to `mtvec.BASE`

> After servicing, `mret` restores `PC = mepc` to resume execution (if recovery is possible).

Multiple Exceptions

Since multiple instructions are **in flight** in a pipeline, more than one could raise an exception at once.
To maintain correctness, RISC-V defines **precise exceptions**, meaning:

- Only the **earliest instruction** (in program order) is handled first.
- All **later** instructions are **flushed**.
- The machine state appears as if instructions were executed **sequentially** up to that fault.

This guarantees that the processor can safely restart or recover from exceptions.

In complex out-of-order pipelines:

- Multiple instructions may issue and complete per cycle.
- Maintaining precise exceptions becomes difficult, requiring **reorder buffers** or **commit logic** to preserve instruction order.

Timer Interrupts in RISC-V

A **Timer Interrupt** occurs when a hardware timer reaches a preset value.
It's an asynchronous interrupt, often used by operating systems to enforce **time-slicing** in multitasking.

Registers

Register	Description
`mtime`	64-bit real-time counter (increments continuously since power-on)
`mtimecmp`	64-bit compare register that sets the next interrupt time

An interrupt occurs when:

> At that instant, the **Machine Timer Interrupt Pending bit (`MTIP`)** in `mip` is set to `1`.

Timer Interrupt Handling Process

1. The hardware sets `MTIP = 1` in the `mip` register.
2. If `MTIE = 1` (enabled in `mie`) and `MIE = 1` (global enable in `mstatus`),
   → Control transfers to the **timer interrupt handler** at `mtvec`.
3. The handler typically:
   - Updates `mtimecmp` for the next tick.
   - Invokes the OS scheduler for time-slice management.
   - Returns via `mret`.

Example:

```
Timer_Handler:
    csrr  x5, mtime
    li    x6, 1000000
    add   x5, x5, x6
    csrw  mtimecmp, x5     # Schedule next interrupt after 1M ticks
    mret
```

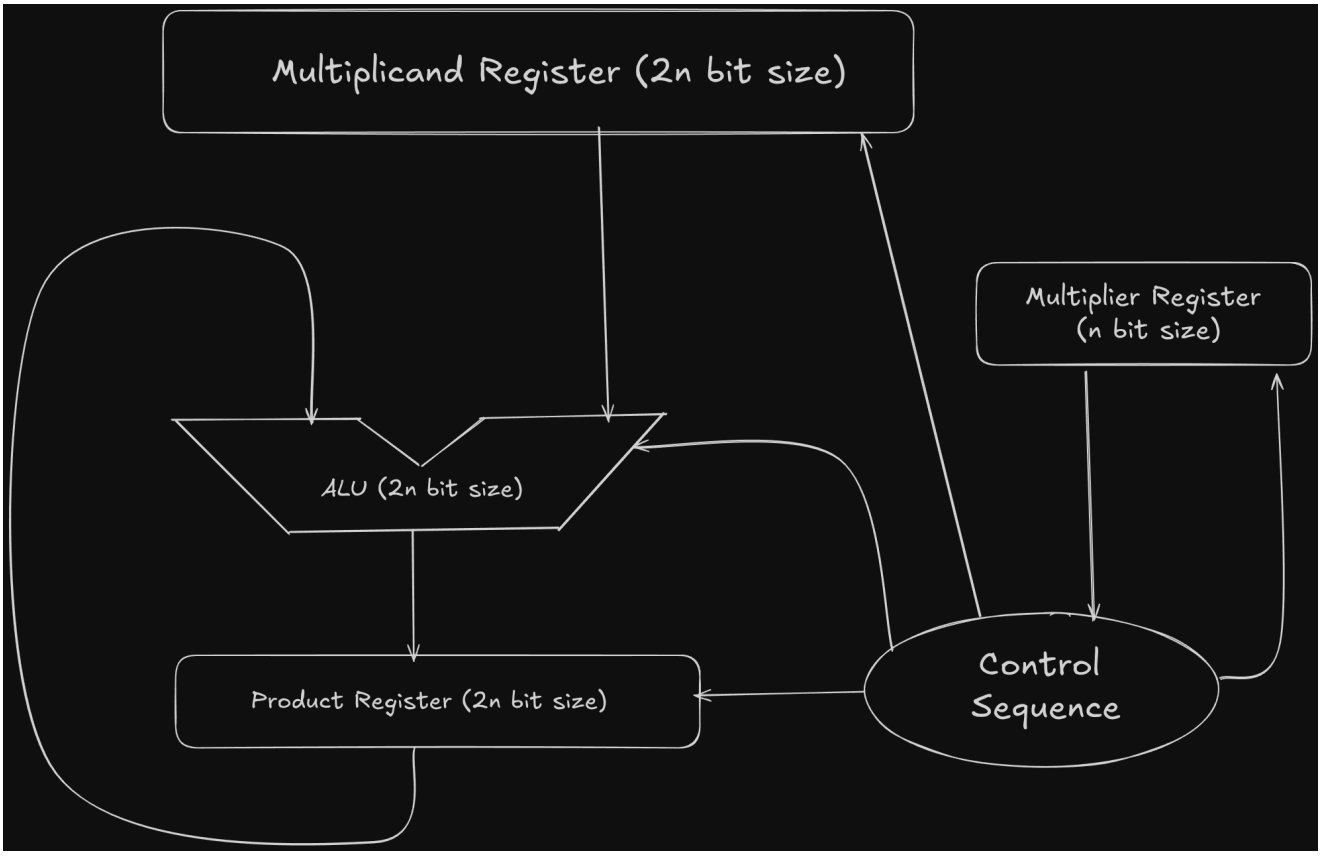
Multiplication: Sequential Circuit Multiplier

The **sequential circuit multiplier** performs multiplication through a series of controlled shift-and-add steps, using a minimal amount of hardware.

Introduction

The hardware includes:

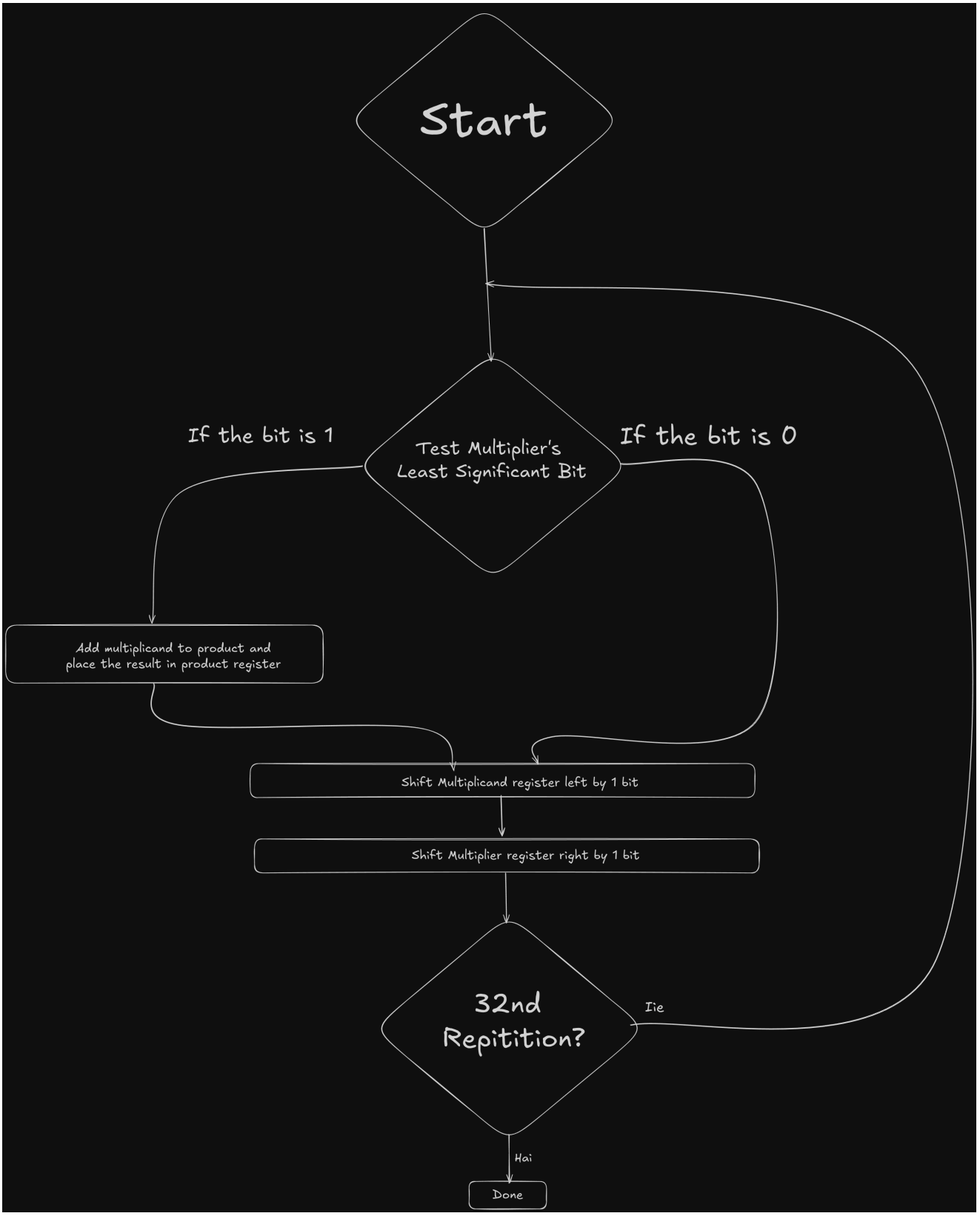
- A **2n-bit Multiplicand Register**
- A **2n-bit Product Register**
- A **n-bit Multiplier Register**
- An **ALU** of 2n-bit size for addition and subtraction
- A **Control Unit** to coordinate shifts and updates



Operation

1. The **32-bit multiplicand** is loaded into the **right half** of the 64-bit Multiplicand register.
2. The **Product register** is initialized to **0**.
3. On each clock cycle:
   - The **least significant bit (LSB)** of the **Multiplier** is checked.
   - If the LSB = 1, the **Multiplicand** is **added** to the **Product register** using the ALU.
   - The **Multiplicand** is then **shifted left** by one bit (×2).
   - The **Multiplier** is **shifted right** by one bit (÷2).

4. This process repeats for 32 iterations — one per multiplier bit.
5. At the end, the **Product register** holds the 64-bit final result.



Notes on Control

- The **control unit** manages when to:
  - Perform an addition.
  - Shift the **Multiplicand** and **Multiplier** registers.
  - Write new values into the **Product** register.

- The algorithm stops after all 32 multiplier bits have been processed.

Multiplication: Refined Sequential Circuit Multiplier

This circuit implements multiplication using a **single n-bit adder** instead of a 2n-bit one, making it compact and efficient.

Operation

- Registers **P** and **Q** are **shift registers**, concatenated to hold the **partial product (PP_i)** and the **multiplier** respectively.
- The least significant bit (q_0) of the multiplier determines the **Add / No-Add** control signal.
  - If $q_o = 1$ → The **multiplicand (M)** is selected by the multiplexer (MUX) and added to the current partial product (PP_i).
  - If $q_o = 0$ → The MUX selects **0**, meaning no addition occurs in that cycle.
- After the addition (if any), the concatenated registers (**P||Q**) are **shifted right by one bit**, preparing for the next bit of the multiplier.
- This process repeats for **n cycles**, one for each bit of the multiplier.

Initialization

- The **Accumulator Register (A)**: holding the initial partial product (PP_0) is loaded with **all zeros** at the start.

Faster Multiplication (Parallel Adders)

A faster multiplication can be achieved by using **parallel hardware** instead of sequential iteration.

Principle

Instead of reusing one adder across all multiplier bits (which takes *n* cycles),
we dedicate **one 32-bit adder for each bit** of the multiplier.

- Each adder stage computes a **partial product**.
- One input to each adder is the **multiplicand** ANDed with the corresponding **multiplier bit**.
- The other input is the **partial product** from the previous stage.

This structure forms a **carry-save adder (CSA) tree** or **array multiplier**, allowing all partial products to be computed **simultaneously**.

Key Points

- Requires **n adders** (e.g., 32 for a 32-bit multiplier).
- Significantly **faster** since it performs additions in parallel.
- Consumes **more area and power** compared to sequential designs.
- Used in high-performance processors where speed outweighs cost.

Comparison of Multiplier Implementations

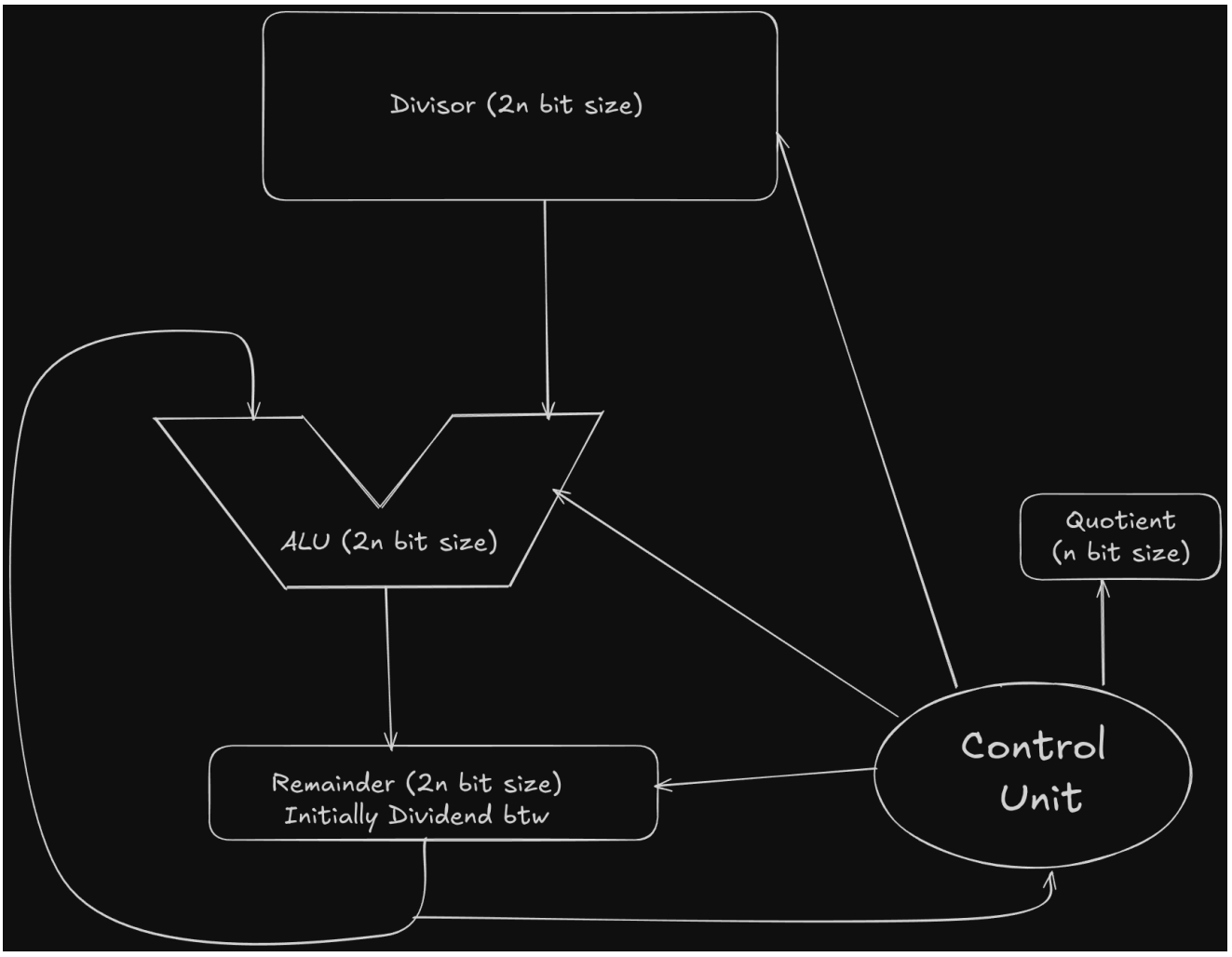
Feature	Sequential (First Version)	Refined Sequential	Parallel (Faster Multiplier)
Adder Used	Single 64-bit ALU reused for all steps	Single *n*-bit adder reused	Multiple *n*-bit adders (one per multiplier bit)
Registers	Multiplicand, Multiplier, Product (P)	Shift registers P & Q for partial products	Parallel array of adders; implicit partial sums
Shifting	Multiplicand ← left, Multiplier → right	P & Q concatenated and shifted	No shifting — all bits processed simultaneously
Cycles Required	*n* cycles (one per multiplier bit)	*n* cycles	1 cycle (fully parallel)

Division: Sequential (Restoring) Algorithm

The **first version** of the hardware division circuit mirrors the multiplication setup, but with a few key differences in control and shifting.

Hardware Setup

- **Divisor**, **ALU**, and **Remainder** registers are all **64 bits (2n bits)** wide.
- The **Quotient register** is **32 bits**.
- **Control logic** manages when to shift and when to write back results.

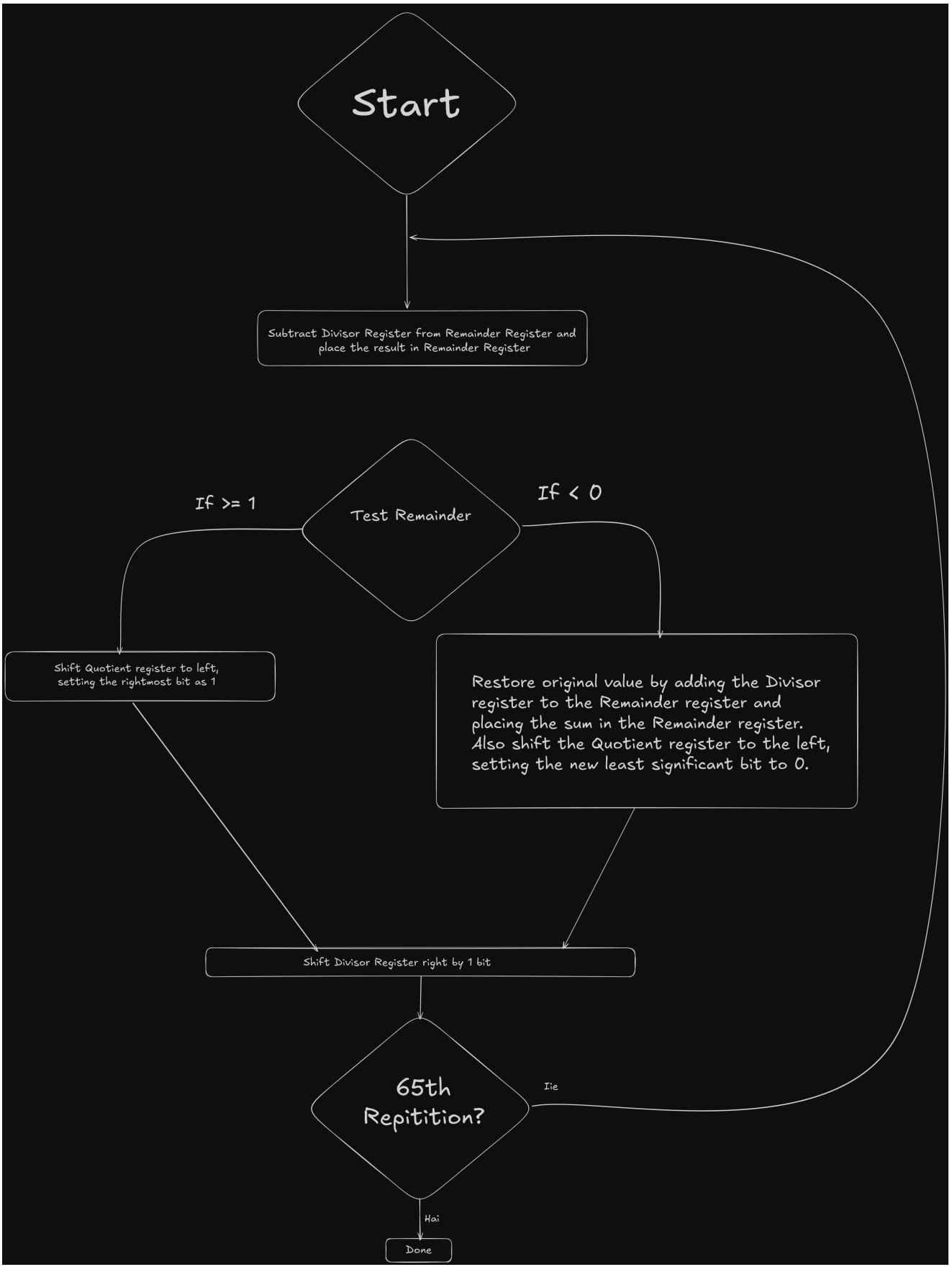


Initialization

1. **Remainder register** ← Dividend
2. **Quotient register** ← 0
3. **Divisor register** ← Divisor placed in **left half** of 64-bit register
   (so it aligns properly as it's shifted right in each step)

Algorithm Operation

- The divisor is **shifted right** by 1 bit each iteration to align with the current remainder.
- The hardware performs **subtraction** to check if the divisor fits into the current remainder.
- If subtraction result ≥ 0:
  - The **remainder** keeps the subtracted value.
  - The corresponding bit in **quotient** is set to **1**.
- Otherwise:
  - The **remainder** is **restored** (previous value kept).
  - Quotient bit set to **0**.



Feature	Description
Cycles Required	(n + 1) steps
Registers Used	Divisor (64-bit), Remainder (64-bit), Quotient (32-bit)
Shifts	Divisor → right each step

Feature	Description
Control Logic	Decides shift timing and writeback
Output	Final **quotient** in Quotient register and **remainder** in Remainder register

---

Division: Improved (Non-Restoring) Version

The improved version optimizes hardware usage by reducing register and ALU width.

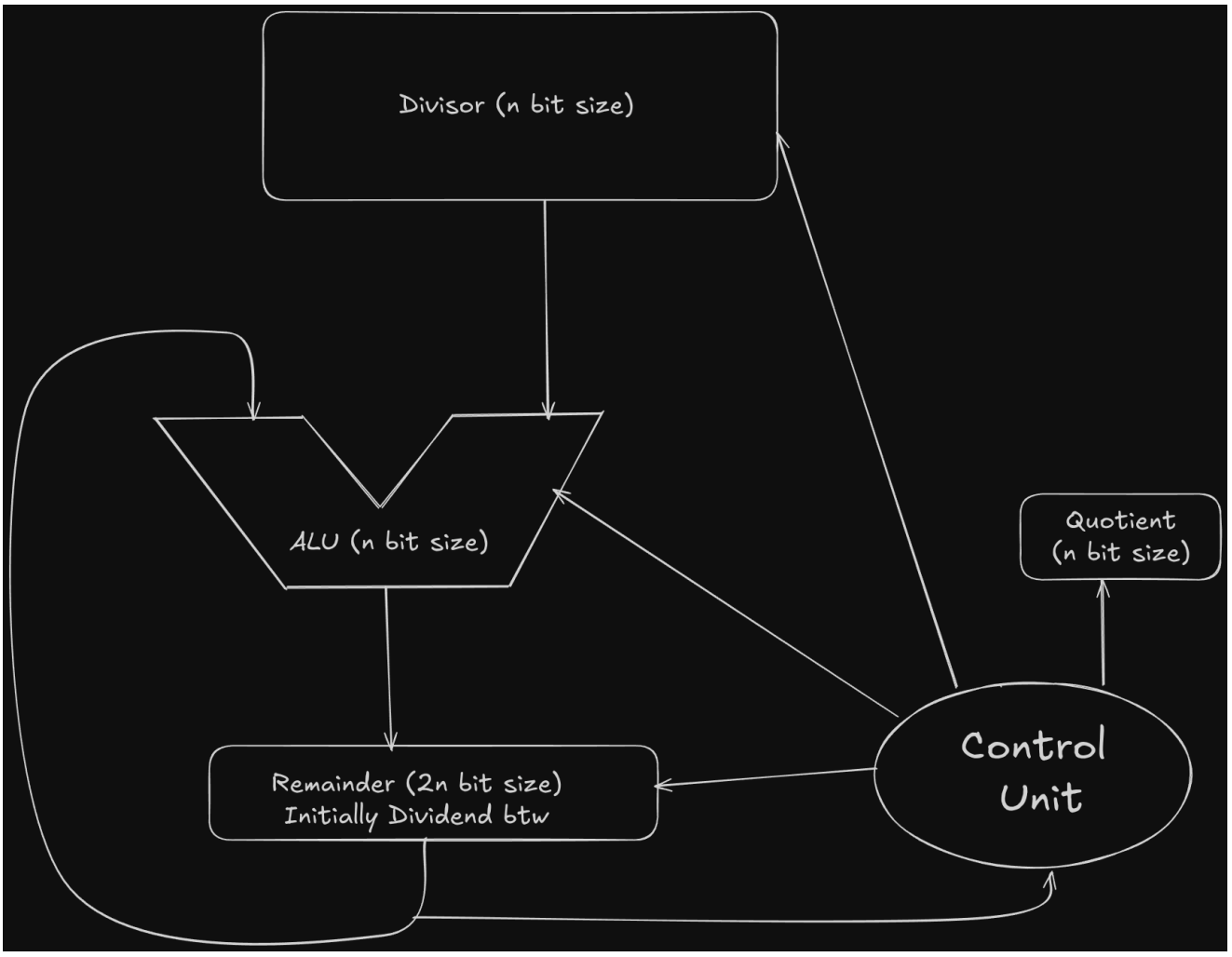
Hardware Setup

Component	Width	Description
Divisor Register	n bits	Holds the divisor
ALU	n bits	Performs subtraction and restoration
Remainder Register	2n bits	Upper n bits initially 0, lower n bits = dividend

Initially,

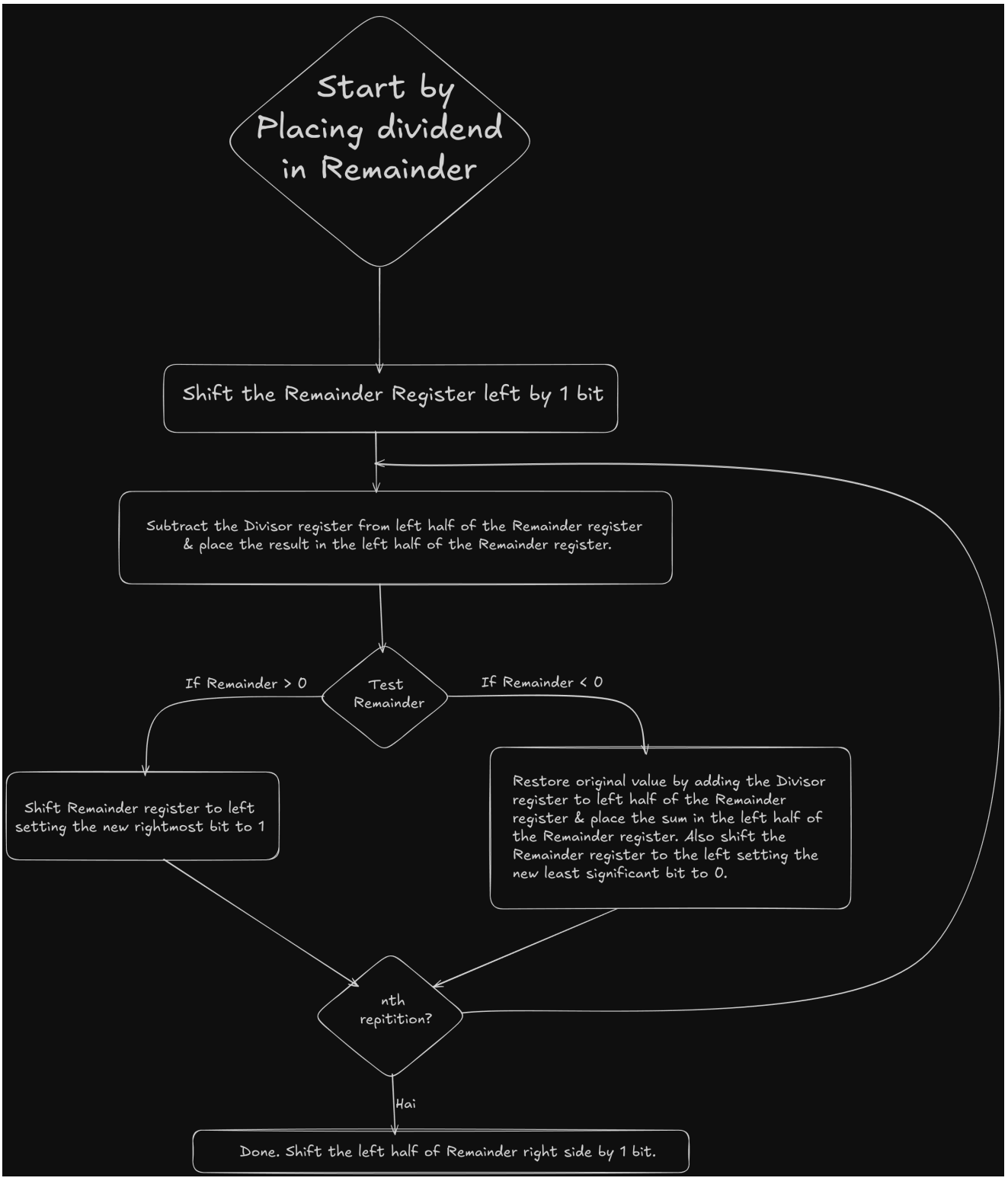
At the end of *n* iterations,

- Upper n bits → **Remainder**
- Lower n bits → **Quotient**



Algorithm Steps

1. **Shift Left**
2. **Subtract Divisor**
3. **Check Sign**
   - If
     then restore:
     and shift left, setting the new LSB = 0
   - Else, set LSB = 1



Advantages

Feature	Improvement
ALU Width	n-bit (instead of 2n-bit)
Divisor Register	n-bit only
Remainder Register	Combined remainder + quotient

Feature	Improvement
Shifting	Remainder shifted left, no separate quotient register needed
Hardware Complexity	Reduced compared to first version

Signed Division Considerations

Before performing division:

1. Record the **signs** of the dividend and divisor.
2. Perform division on their **absolute values**.
3. Negate the **quotient** if signs differ.
4. Adjust the **remainder** sign such that the following always holds:

Dividend	Divisor	Quotient	Remainder
+	+	+	+
-	+	-	-
+	-	-	+
-	-	+	-

RISC-V “M” Extension Instructions

Instruction	Description	Operation
`MUL`	Multiply (low 32 bits)	
`MULH`	Multiply high (signed × signed)	
`MULHSU`	Multiply high (signed × unsigned)	
`MULHU`	Multiply high (unsigned × unsigned)	
`DIV`	Divide (signed)	
`DIVU`	Divide (unsigned)	
`REM`	Remainder (signed)	
`REMU`	Remainder (unsigned)	

Floating-Point Numbers & Operations (IEEE 754)

Floating-point exists because fixed-width integers are hopeless when you need to represent numbers that are ridiculously tiny or astronomically huge.
Binary scientific notation saves the day.

Think of a number like:

-
- We can never fit these elegantly using plain integers. IEEE 754 gives a portable, standardized way to represent such values.

IEEE 754 Floating-Point Representation

There are multiple formats. The common ones:

- **Single precision**: 32-bit
- **Double precision**: 64-bit

Both follow the same conceptual structure:

```
| Sign | Exponent (biased) | Mantissa |
```

Single Precision (32-bit)

```
31        30           23 22                    0
+----------+-------------+----------------------+
|   Sign   |  Exponent   |      Mantissa        |
+----------+-------------+----------------------+
|   1 bit  |   8 bits    |       23 bits        |
```

Components

Sign bit (S)

- 0 → positive
- 1 → negative

Exponent (E') — 8 bits
Stored using *excess-127* representation:

Valid “normal” exponents use are reserved for special cases (zero, subnormals, Inf, NaN).

Fraction (mantissa)
Only the fractional part is stored:

```
Actual significand = 1.M
```

That leading **1** is *implicit*. So precision = **24 bits**, not 23.

Value Represented

> Zero is special: it has no leading 1, so exponent field = 0.

Double Precision (64-bit)

```
63         62                  52 51                   0
+------------+------------------+----------------------+
|    Sign    |    Exponent      |      Mantissa        |
+------------+------------------+----------------------+
|    1 bit   |     11 bits      |       52 bits        |
```

Exponent uses excess-1023 representation:

Normal exponent range: Actual exponent range: **Significand precision:**
1 implicit bit + 52 stored bits = **53-bit precision** (~16 decimal digits).

Precision vs Range

- More **mantissa bits** → more precision (more significant digits).
- More **exponent bits** → wider numeric range.

Double precision increases both.

Overflow & Underflow

Overflow: exponent too large → value can't be represented.
Underflow: exponent too negative → too close to zero to represent.

Some architectures raise exceptions for these events.
RISC-V chooses a calmer path:

- No trap is raised.
- Software checks the **FCSR** (Floating-Point Control and Status Register) to detect overflow/underflow.

This keeps hardware simpler and pipelines happier.

Extra Notes

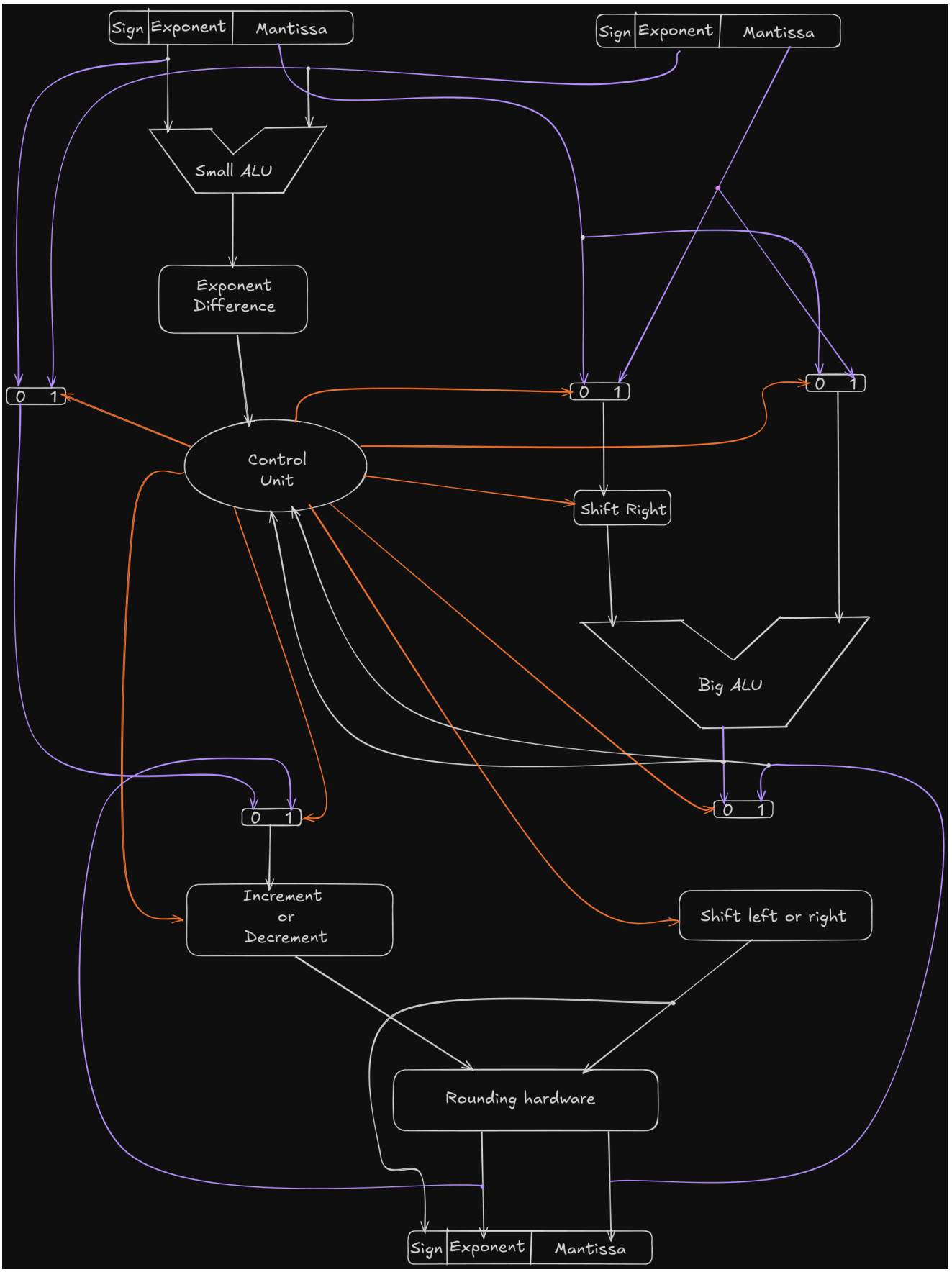
- IEEE754 packs numbers very efficiently by omitting the leading 1 for normal numbers.
- Zero and subnormal numbers *do not* have that implicit 1.
- Infinity and NaN come from exponent field = all ones.

Round the significand to appropriate number of bits.

ADDITION AND SUBTRACTION (Concept + Hardware View)

Floating-point addition and subtraction follow a strict sequence of steps because the numbers are stored in scientific-notation form:

The goal is to make the exponents match, add the significands, then renormalize and round.



Step 1: Align Exponents

Determine which operand has the **smaller exponent**.

Let the exponents be Find the difference:

Shift the **mantissa of the smaller-exponent number** to the right by nnn positions. This reduces its contribution so both operands now represent quantities at the same scale.

Example if n = 2:
The smaller mantissa becomes

(two-bit right shift).

Step 2: Add or Subtract Mantissas

Once exponents match, the hardware's main adder (the “big ALU”) performs the significand addition:

Sign bits decide addition vs subtraction.

Meanwhile:

• The exponent of the **larger operand** becomes the tentative exponent of the result.
• ALU and multiplexors ensure the hardware selects
1. the larger exponent,
2. the shifted smaller mantissa,
3. the larger mantissa.

Step 3: Normalize the Result

The sum might not be in normalized form.
Two possibilities:

• If the sum is **≥ 2**, shift right once and increment exponent.
• If the sum is **< 1**, shift left until it reaches the 1.xxx1.xxx1.xxx normalized form and decrement exponent accordingly.

The hardware checks for these conditions automatically.

If the final exponent is within the allowed range:

there is no overflow or underflow.
In biased form:

Step 4: Rounding (Single-Precision)

Single-precision stores only **23 fraction bits**.
The normalized result typically has extra bits that must be rounded.

IEEE-754 uses three key bits beyond the 23rd:

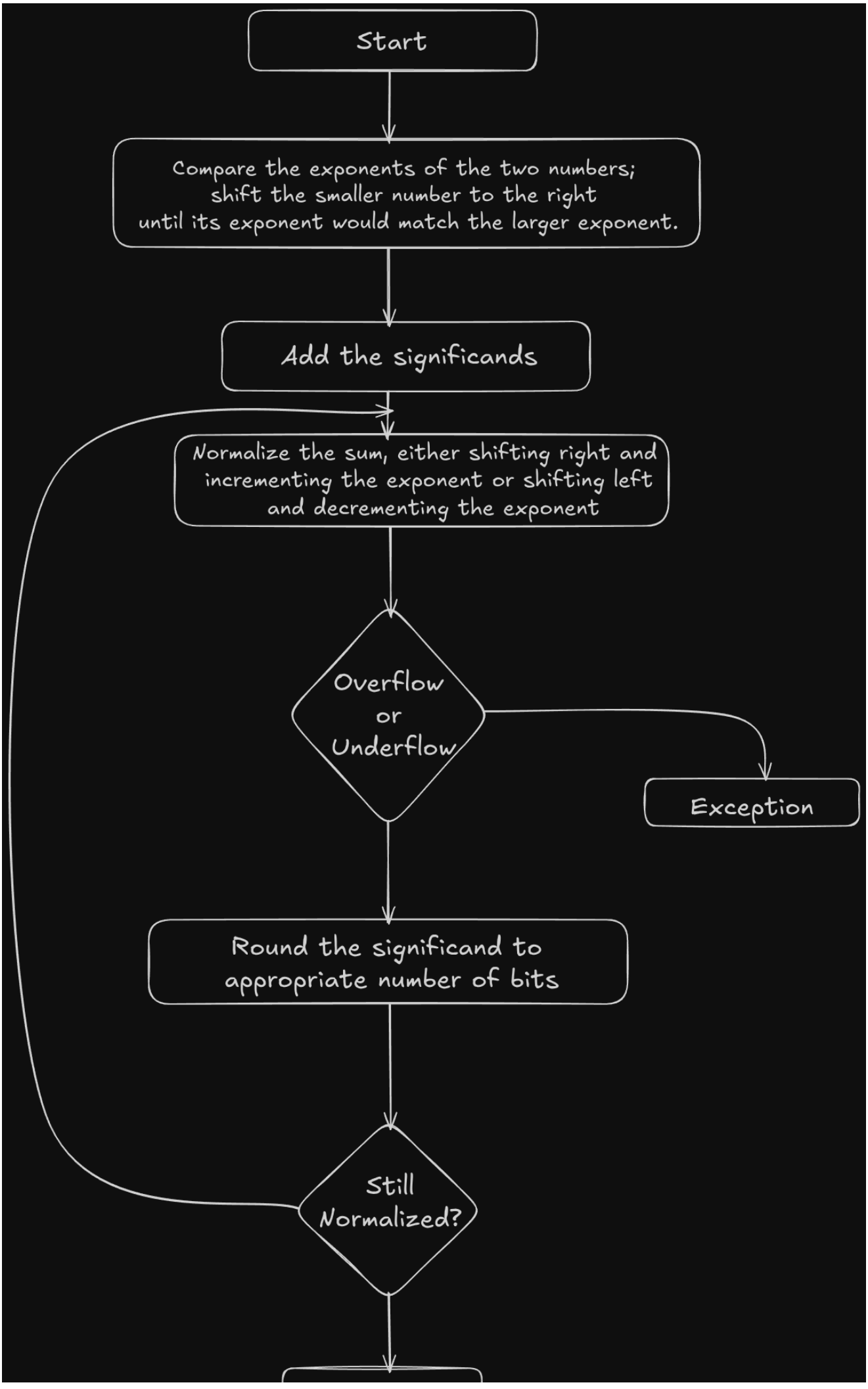
• **Guard bit (G)** – immediately after the stored LSB
• **Round bit (R)** – next bit

• **Sticky bit (S)** – OR of all remaining bits after R

Your notes focus on R and S, but the idea is the same.
The rounding rule applied is typically “round to nearest, ties to even.”

Sticky bit matters because any leftover 1 makes the tail nonzero.

If rounding overflows the mantissa (e.g., becomes 10.000…10.000…10.000…), normalization runs again and increments exponent.


Start
Compare the exponents of the two numbers; shift the smaller number to the right until its exponent would match the larger exponent.
Add the significands
Normalize the sum, either shifting right and incrementing the exponent or shifting left and decrementing the exponent
Overflow or Underflow
Exception
Round the significand to appropriate number of bits
Still Normalized?

A Tiny Example

Let’s add:

Step 1: Align exponents

Difference in exponents:

Shift mantissa of BBB right by 2:

Now both use exponent `4`

Step 2: Add mantissas

Step 3: Normalize

Already normalized `(≥1 and <2)`.
Exponent stays `4`.

Step 4: Rounding

Assume we need only **4 fraction bits** in this toy example:

Result mantissa before rounding:

Fraction to keep: **1100**
Guard = 1
Round = 0
Sticky = 10 OR → 1

Tail is nonzero → round up:

Final result:

MULTIPLICATION (IEEE-754 Floating Point)

Let two floating-point numbers be

Step 1: Add the Exponents

If exponents are *unbiased*:

If exponents are *biased* (single precision):

Then

Bias got added twice, so subtract it once to keep the result correctly biased.

Step 2: Multiply the Mantissas and Compute Sign

Mantissa:

Sign:

Step 3: Normalize (If Needed)

If

the mantissa is already normalized.

If

shift right by one and increment exponent.

If

shift left and decrement exponent until normalized.

Example (Single Precision)

Multiply:

1. Exponent Addition

Unbiased:

Biased (just to show the mechanism):

2. Mantissa Multiply

Sign:

3. Normalization

Already normalized, no shifting needed.

Final Result

Conceptually, IEEE-754 fields would be:

- Sign = 0
- Exponent (biased) = 128
- Mantissa = fractional part of (1.875)

Rounding and Truncation in Floating-Point Arithmetic

When performing floating-point arithmetic, intermediate results may produce **more mantissa bits** than the format can store.
Single precision stores only **23 fraction bits** (with an implied leading 1).
Extra bits are handled using **truncation** or **rounding**.

Extra bits usually include:

- Guard bit G
- Round bit R
- Sticky bit S (OR of all bits after R)

1. Chopping (Truncation)

Idea:
Simply discard all extra bits.

Example (keeping 3 bits):

Exact:
Chopped:

Error range:
From 0 to almost 1 ulp.

Issue:
Chopping is **biased**, errors always lean in the same direction.

2. Von Neumann Rounding (Unbiased Truncation)

Rules:

- If **all discarded bits = 0**, truncate normally.
- If **any discarded bit = 1**, set the **LSB of retained bits** to 1.

Example (keeping 3 bits):

If discarded bits

If any discarded bit = 1:

Why:
This gives **unbiased** results, positive and negative errors cancel over many operations.

3. IEEE-754 Rounding to Nearest (Ties to Even)

This is the default rounding mode in IEEE-754.

Uses **G, R, S** bits.

Case A: No rounding

If

just drop the bits.

Example:

Case B: Normal round up

If

add 1 to the LSB:

Case C: Tie case

If

this number is **exactly halfway**.

Use the **guard bit GGG** to break the tie:

- G = 0 → truncate (keep LSB even)
- G = 1 → round up (make mantissa even)

Prevents systematic bias.

Example (Rounded Step-by-Step)

Suppose we keep 3 mantissa bits.
Exact internal mantissa:

Where:

- Guard G = 1
- Round R = 0
- Sticky S = 1

Since R = 0 and S = 1 → discarded part > half → **round up**.

If this overflows to something like `10.x`, the mantissa must be renormalized.

RISC-V floating-point assembly language

Arithmetic	FP add single	`fadd.s f0, f1, f2`	`f0 = f1 + f2`	FP add (single precision)
	FP subtract single	`fsub.s f0, f1, f2`	`f0 = f1 - f2`	FP subtract (single precision)
	FP multiply single	`fmul.s f0, f1, f2`	`f0 = f1 * f2`	FP multiply (single precision)
	FP divide single	`fdiv.s f0, f1, f2`	`f0 = f1 / f2`	FP divide (single precision)
	FP square root single	`fsqrt.s f0, f1`	`f0 = √f1`	FP square root (single precision)
	FP add double	`fadd.d f0, f1, f2`	`f0 = f1 + f2`	FP add (double precision)
	FP subtract double	`fsub.d f0, f1, f2`	`f0 = f1 - f2`	FP subtract (double precision)
	FP multiply double	`fmul.d f0, f1, f2`	`f0 = f1 * f2`	FP multiply (double precision)
	FP divide double	`fdiv.d f0, f1, f2`	`f0 = f1 / f2`	FP divide (double precision)
	FP square root double	`fsqrt.d f0, f1`	`f0 = √f1`	FP square root (double precision)
Comparison	FP equality single	`feq.s x5, f0, f1`	`x5 = 1 if f0 == f1, else 0`	FP comparison (single precision)
	FP less than single	`flt.s x5, f0, f1`	`x5 = 1 if f0 < f1, else 0`	FP comparison (single precision)
	FP less than or equals single	`fle.s x5, f0, f1`	`x5 = 1 if f0 <= f1, else 0`	FP comparison (single precision)
	FP equality double	`feq.d x5, f0, f1`	`x5 = 1 if f0 == f1, else 0`	FP comparison (double precision)
	FP less than double	`flt.d x5, f0, f1`	`x5 = 1 if f0 < f1, else 0`	FP comparison (double precision)
	FP less than or equals double	`fle.d x5, f0, f1`	`x5 = 1 if f0 <= f1, else 0`	FP comparison (double precision)
Data transfer	FP load word	`flw f0, 4(x5)`	`f0 = Memory[x5 + 4]`	Load single-precision from memory
	FP load doubleword	`fld f0, 8(x5)`	`f0 = Memory[x5 + 8]`	Load double-precision from memory
	FP store word	`fsw f0, 4(x5)`	`Memory[x5 + 4] = f0`	Store single-precision from memory
	FP store doubleword	`fsd f0, 8(x5)`	`Memory[x5 + 8] = f0`	Store double-precision from memory

The floating-point extension in RISC-V basically gives the CPU the provision for dealing with real numbers. We get 32 floating-point registers (`f0–f31`) and a bunch of instructions that follow IEEE-754 rules.

Arithmetic instructions like `fadd.s` or `fmul.d` always need the `.s` or `.d` suffix because the hardware treats 32-bit and 64-bit floats differently. Loads and stores don't need that suffix since `flw` / `fsw` or `fld` / `fsd` already tell the CPU how wide the data is; they just shuttle values between memory and the `f` registers with no math involved.

Comparisons (`feq.s` , `flt.d` , etc.) take two floats, apply IEEE-754 comparison rules, and drop the result into an integer register as a clean 0/1.

Compressed Instructions (RVC)

The C extension provides **16-bit encodings** for the most frequently used 32-bit instructions. These compressed forms slot naturally into normal RISC-V code; the CPU happily mixes 16-bit and 32-bit instructions, since 32-bit ones can now begin at any 16-bit boundary.

The idea is simple: many instructions don't need the full 32-bit space. If the offset is small, the immediate is tiny, or you're using common registers like `x0` , `x1` (link), `x2` (stack pointer), or the popular "compressed register subset" (`x8–x15`), then a compact form does the job.

Instruction Encoding Philosophy

There are **eight** compressed formats, all designed around one rule of sanity:
the **two source registers** stay in consistent bit positions across formats.
The destination register may wander a bit, but when a full 5-bit rd is present, it remains in the same place as in the original 32-bit encoding.

Immediates behave as in the main ISA: sign-extended from bit 12, and intentionally scrambled to cut down mux complexity in hardware.

Load and Store Instructions in RVC

Compressed load/store instructions try to stretch their reach using **scaled offsets**:

- Word loads/stores scale offsets by **4**
- Doubleword by **8**
- Quadword by **16**

RVC provides **two families** of load/store instructions:

1) Stack-Pointer-Based Loads/Stores (SP-based)

These use `x2` (the stack pointer) as the base. They can target any data register.
All of these instructions use the **CI format**.

Examples:

- **C.LWSP**
  Loads a 32-bit value into `rd` from `x2 + offset×4`.
  Expands to: `lw rd, offset[7:2](x2)`
- **C.LDSP** (RV64C/RV128C)
  Loads 64-bit from `x2 + offset×8`.
  Expands to: `ld rd, offset[8:3](x2)`
- **C.FLWSP** (RV32FC only)
  Loads an FP single from `x2 + offset×4`.
  Expands to: `flw rd, offset[7:2](x2)`
- **C.FLDSP** (RV32DC / RV64DC)
  Loads an FP double from `x2 + offset×8`.
  Expands to: `fld rd, offset[8:3](x2)`

These instructions exist because stack accesses are overwhelmingly common, so shrinking them gives excellent density Ws.

2) Register-Based Loads/Stores (CL/CS formats)

These use one of the eight “compressed registers” (`x8–x15`) as the base, and another from the same set as data register.
This limitation is part of how RVC fits everything into 16 bits.

- **C.LW** — CL format
  Loads a 32-bit word using scaled offset×4.
  Expands to: `lw rd, offset[6:2](rs1)`
- **C.SW** — CS format
  Stores a 32-bit word using scaled offset×4.
  Expands to: `sw rs2, offset[6:2](rs1)`

These don’t involve the stack pointer; they target general memory but only with the compressed register set.

If this helped you, treat me with a dosa.